

Vericoding with Axiomander

Specification is the control surface.

Code is a derived artifact.

Correctness requires proof.

Part I

Why Vericoding?

The optimism problem

LLMs are **trained to satisfy the prompt**, not to be correct.

- They produce code that *looks* like a solution to the request.
- They are systematically **optimistic**: edge cases, error paths, and invariants get glossed.
- "It compiles and the happy path runs" is treated as success.
- The failure mode is silent: plausible code that is subtly wrong.

The model optimises for **apparent** intent satisfaction. We need a way to test **actual** intent satisfaction — **deterministically**.

The ladder of guarantees

How strongly can we pin down intent?

Mechanism	Catches	Misses
Prompt review	gross misunderstanding	everything subtle
Types	shape errors	values, relations, invariants
Tests	the cases you thought of	the cases you didn't
Strong specification	anything expressible as a property	only what you leave unstated

Types and tests are *samples* of intent. A **strong specification** makes intent **explicit and mechanically checkable** over *all* inputs — and it can be **proved**, not sampled.

Tests sample. Proofs quantify.

A test

```
def test_clamp():  
    assert clamp(5, 0, 10) == 5  
    assert clamp(-3, 0, 10) == 0  
    assert clamp(99, 0, 10) == 10
```

A finite set of points. Green means *these three* are right.

A specification

$\forall v, lo, hi. lo \leq hi \Rightarrow lo \leq \text{clamp}(v, lo, hi) \leq hi$

A statement over **all** inputs. Proved means *every* input is right.

A passing test is evidence. **A proof is a guarantee.**

...a guarantee **relative to the specification**. Proof establishes correctness w.r.t. the spec, not w.r.t. unstated intent — so the spec itself becomes the artifact we must get right.

The spec is now the thing to get right

Proof moves the burden, it doesn't remove it. Correctness is **relative to the specification**, so a spec can still:

- omit a requirement, or encode a wrong assumption;
- over-constrain (reject valid behaviour) or under-constrain (allow wrong behaviour).

This is a feature of *where the work goes*, not a hole. The reviewable surface shrinks from "all the code" to "the contracts" — and the contracts are small, declarative, and the explicit object of review.

Future work: tooling to test the *meaning* of a specification — exercising it with examples, checking satisfiability and non-vacuity, surfacing over/under-constraint — potentially **before** any code is written.

What "vericoding" means

A development loop in which:

1. The human writes and iterates on a **strong specification**.
2. An LLM proposes an **implementation** to satisfy it.
3. A **verifier** decides — deterministically — whether the code meets the spec.
4. On failure, the verifier returns a **structured residual** that drives the next attempt.

The spec is the thing you maintain. The code is generated, checked, and regenerated underneath it.

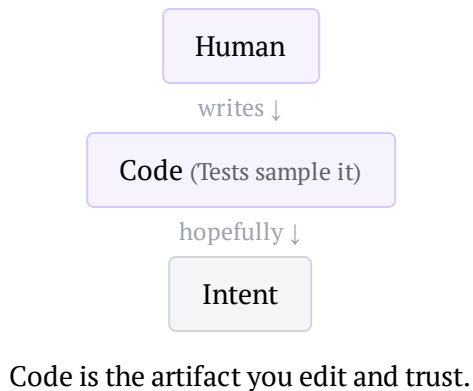
Part II

The Methodology

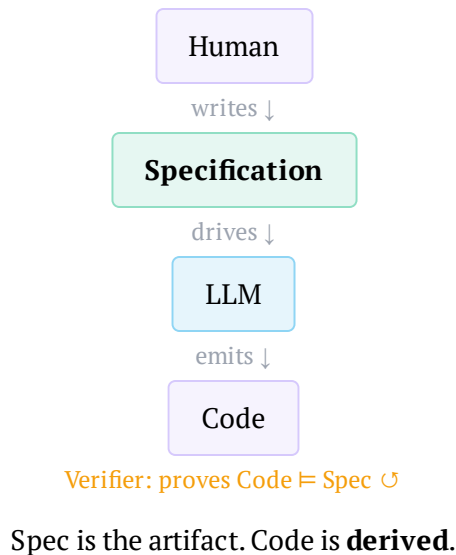
Specification is the control surface

You steer the system by editing the **specification**, not the code.

Traditional



Vericoding



Code is a derived artifact

- The implementation is **disposable**. If it doesn't verify, throw it away and regenerate.
- The spec persists across many implementations.
- This inverts the usual trust relationship: we don't trust the code because we read it — we trust it because it was **proved** against a spec we *did* read.
- The reviewable surface shrinks from "all the code" to "the contracts at the boundary."

$$\underbrace{(P, Q)}_{\text{spec, human-authored}} \rightsquigarrow \underbrace{c}_{\text{code, LLM-derived}} \quad \text{such that} \quad \models \{P\} c \{Q\}$$

The human authors **both** halves of the spec: the assumption P on inputs and the guarantee Q on outputs. The code is derived to make Q hold **under** P — we supply only P -constrained inputs.

Correctness requires proof

A specification is only as good as our ability to **discharge** it.

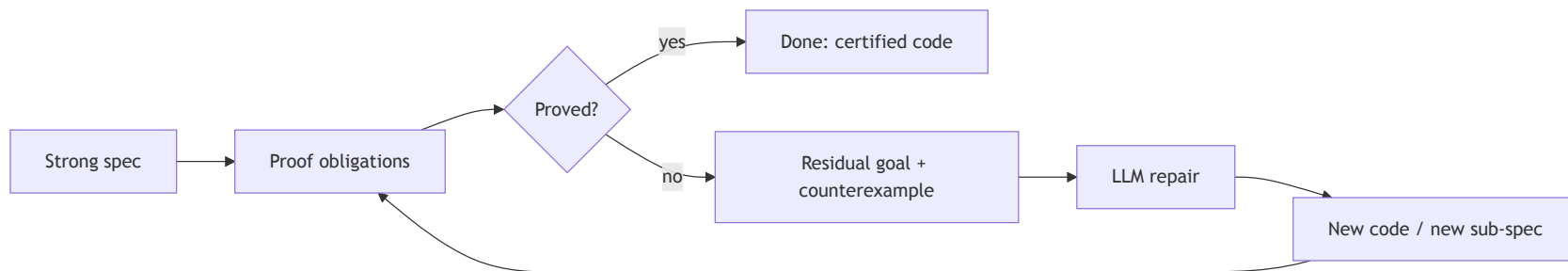
- Runtime contracts catch violations *after* deployment, on inputs you happened to hit.
- Static proof catches violations *before* deployment, on **all** inputs.
- Axiomander reduces "does the code meet the spec?" to a **Rocq proof obligation** — a binary, deterministic verdict.

$$\{P\} c \{Q\} \equiv \forall s. P(s) \Rightarrow \text{wp}(c, Q)(s)$$

Here $\text{wp}(\mathbf{c}, \mathbf{Q})$ is the **weakest precondition** (WP) calculus — the weakest predicate that must hold before command c so that postcondition Q holds after. The Hoare triple holds iff the precondition P implies $\text{wp}(\mathbf{c}, \mathbf{Q})$. Proved sound in Rocq.

The heal-loop

The verifier doesn't just say "no" — it says **why**, in a form the LLM can act on.



- A counterexample is a **concrete witness** of failure — typed values, not vibes.
- A residual goal is the **exact remaining obligation** with its hypotheses.
- The LLM repairs against the residual, not the original prompt.

Why the heal-loop beats re-prompting

Naive: "try again"

```
LLM: here is code
human: it's wrong
LLM: here is different code
human: still wrong
...
```

No signal. The model guesses again.

Heal-loop: drive on residual

```
Goal:
  lo <= result <= hi
Hyp:
  val > hi
Counterexample:
  result = val  (= 99, > hi)
```

The model gets the **failing fact**. It fixes *that*.

Determinism turns "convince a reviewer" into "close a goal."

End-to-end: one turn of the loop

1. Human writes the spec

```
def clamp(v: int, lo: int, hi: int) -> int:
    assert lo <= hi                # pre
    ...
    assert lo <= result <= hi      # post
    return result
```

2. LLM emits code (optimistic, wrong)

```
if v < lo: result = lo
else:      result = v      # forgot the hi case!
```

3. Verifier returns a residual + counterexample

```
Goal:  lo <= result <= hi
Hyp:   v > hi
CEX:   v=99, lo=0, hi=10 -> result=99 > hi
```

4. LLM repairs against the failing fact

```
if v < lo:    result = lo
elif v > hi:  result = hi    # added
else:        result = v
```

5. Re-verify -> 6. Qed **certified**

The model never saw the prompt again — only the residual goal and the counterexample. That is the whole loop.

Freedom in sub-specification

To prove the **entry point**, the LLM must specify the **helpers** it introduces.

- The human writes the top-level contract — the *one surface of understanding*.
- The LLM is free to decompose: introduce helper functions, each with **its own** contract.
- Those sub-specifications are the LLM's degrees of freedom — they are how it makes the global proof go through.
- Sub-specs are checked the same way. There is no privileged, unverified layer.

$$\frac{\{P_g\} g \{Q_g\} \quad \{P_f\} f \{Q_f\} \text{ using } Q_g}{\{P_f\} f \{Q_f\}}$$

The caller's proof uses only the callee's **contract** Q_g , never its body.

Sub-specification is free — and self-correcting

"What stops the LLM inventing vacuous or weak helper contracts?" Nothing — and it doesn't matter.

- Sub-specification is **completely free**: the LLM may write any helper contracts it likes.
- A **weak** sub-spec simply *fails to prove the control spec* — unless that helper was irrelevant to it. Bad sub-specs do no damage; they just don't close the goal.
- So the LLM is **incentivised toward strong sub-specs**: stronger lower-level guarantees are what make the weaker, higher-level goal provable.

The control specification is the gatekeeper. It cannot be discharged by leaning on under-powered sub-specs — the proof obligation forces honesty all the way down.

Composability enables scaling

If every edit re-checked the whole program, vericoding would not scale.

- **Iteration cost.** You change one spec; you should re-verify *only* what depends on it.
- **Stubbing.** A library you can't (or won't) verify is replaced by its **contract**. Callers prove against the stub's axioms.
- **Locality of trust.** A function's body can change freely as long as its contract is stable — callers are untouched.

Body changes

invalidate local proofs

Contract changes

invalidate callers

Composition, formally

A contract is an **interface**; a proof is a **cached build artifact**.

$$\text{summary}(g) = \text{hash}(\text{pre}, \text{post}, \text{reads}, \text{writes}, \text{raises})$$

- Caller f depends on $\text{summary}(g)$ – **not** on g 's implementation.
- g 's body changes, $\text{summary}(g)$ stable $\Rightarrow f$'s proof is **reused**.
- $\text{summary}(g)$ changes $\Rightarrow$ re-verify f and its transitive callers only.

The verifier becomes a **build system for correctness**, not a batch theorem prover.

Stubbing libraries as axioms

You don't verify `dict.get`. You **declare its contract** and prove against it.

```
# stubs/builtins.pyi - the contract, not the implementation
def get(d: dict, k, default):
    """
    axiomander:
        ensures:
            implies(k in d, result == d[k])
            implies(k not in d, result == default)
        modifies:
            none
    """
```

- The stub's `ensures` becomes an **axiom** available to callers.
- The trust boundary is **explicit and small**: it's the stub, written once, reviewed once.
- Replacing optimism ("the LLM probably calls `get` correctly") with an enforced contract.

Part III

The Axiomander Architecture

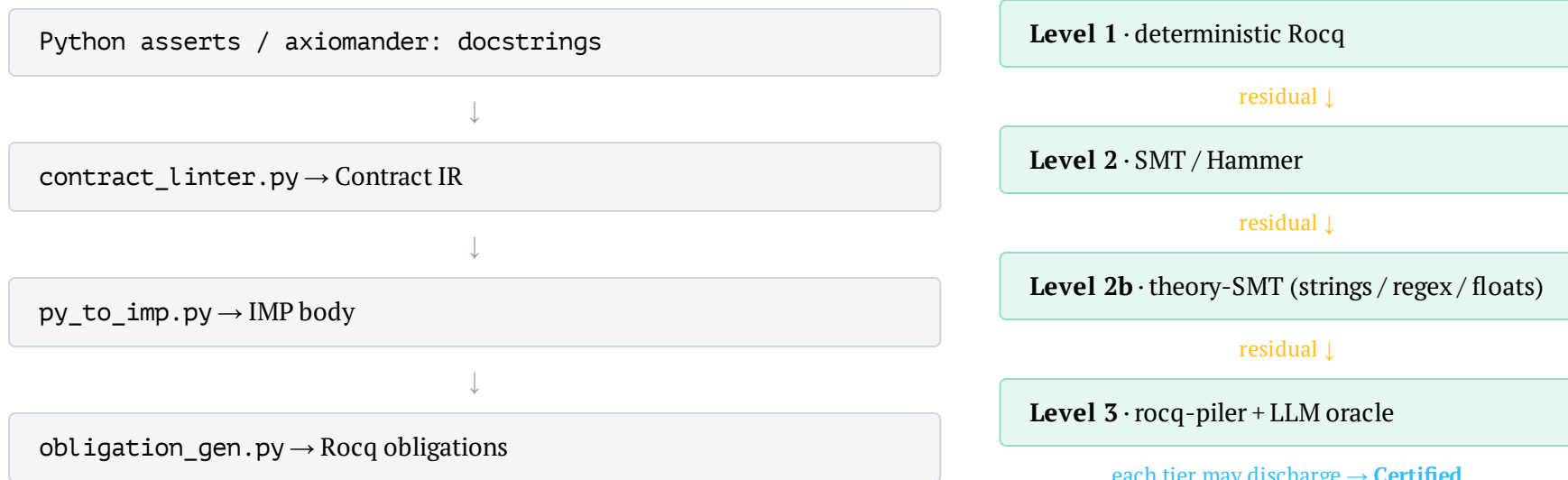
The stack

$$\mathbf{Axiomander} = \text{LLM} + \text{Rocq} + \text{Iris} + \text{Python}$$

Layer	Role
Python	source language — contracts are plain <code>assert</code> / <code>axiomander:</code> docstrings
Rocq	the proof assistant (formerly Coq) — the trust kernel , final authority
Iris	on Rocq — useful for separation logic, resource management, concurrency
LLM	the oracle — closes residual goals, proposes code & sub-specs, never the trust kernel

Rocq is one possible backend. Vericoding is backend-agnostic — any assistant that returns a sound verdict + a structured residual works.

From Python to a proof obligation



Contracts are plain assert statements or verifier-only `axiomander: docstring` blocks — zero imports, zero decorators. Cheap deterministic tactics run first; the LLM is the fallback.

Contracts as ordinary Python

No decorators. No imports. The user's code stays dependency-free.

```
def clamp(val: int, lo: int, hi: int) -> int:
    assert lo <= hi                                # precondition
    if val < lo:    result = lo
    elif val > hi:  result = hi
    else:          result = val
    assert lo <= result <= hi                      # postcondition
    assert implies(val < lo, result == lo)
    assert implies(val > hi, result == hi)
    return result
```

- Leading asserts → preconditions. Trailing asserts → postconditions. Loop-body asserts → invariants.
- The richer `axiomander: docstring` carries `requires / ensures / reads / modifies / raises / units`.
- The linter classifies by **position** and lowers to the Contract IR.

The intermediate language: IMP

We don't prove properties of arbitrary Python. We lower a verified subset to **IMP**.

$$e ::= n \mid x \mid e_1 + e_2 \mid e_1 < e_2 \mid \dots$$
$$\begin{aligned} c ::= & \text{skip} \mid x := e \mid c_1; c_2 \\ & \mid \text{if } e \text{ then } c_1 \text{ else } c_2 \\ & \mid \text{while } e \text{ inv } I \text{ do } c \\ & \mid \text{assert } P \end{aligned}$$

- Values: `VZ` | `VBool` | `VUnit` plus structural `VList` | `VTuple` | `VDict`.
- Mutation works on a heap representation that **parallels** the immutable value.
- IMP is small enough to formalise and prove sound; rich enough to cover the verified subset.

The supported Python fragment

A **large** fragment — large enough to instruct the LLM to write within it, and to **retroactively annotate** many existing Python functions.

Supported now

- assignments, `if / elif / else`
- `while`, `for ... in range`
- function calls + frames
- `list / dict / set`, `str`, `bytes`
- `int / float / bool / None`
- exceptions (raise as outcomes)
- `isinstance` dispatch
- classes / Pydantic fields

On the roadmap

- **concurrency** — separation logic is built for it
- **generators** — coinductive proofs at the generator, **composed** at call sites
- closures, richer iterators
- more of the stdlib via stubs
- wider numeric / string theory

Resists a contract

- reflection / introspection
- monkey-patching, `eval`
- truly arbitrary dynamic dispatch

Most of Python is reachable. The hard cases (concurrency, generators) are **roadmap**, **not walls** — they compose into the same contract framework. Only genuinely non-analysable behaviour resists a contract, and it is **rejected**, **not silently mis-modelled**.

Weakest precondition (WP) calculus

The single source of truth — proven sound in Rocq. $\text{wp}(c, Q)$ computes the weakest precondition of command c w.r.t. postcondition Q , by structural recursion on c :

$$\text{wp}(\text{skip}, Q) = Q$$

$$\text{wp}(x := e, Q) = Q[x \mapsto e]$$

$$\text{wp}(c_1; c_2, Q) = \text{wp}(c_1, \text{wp}(c_2, Q))$$

$$\text{wp}(\text{if } e \text{ then } c_1 \text{ else } c_2, Q) = (e \Rightarrow \text{wp}(c_1, Q)) \wedge (\neg e \Rightarrow \text{wp}(c_2, Q))$$

$$\text{wp}(\text{while } e \text{ inv } I \text{ do } c, Q) = I$$

For loops, the invariant generates two verification conditions:

$$\text{VC}_1 : I \wedge \neg e \Rightarrow Q$$

$$\text{VC}_2 : I \wedge e \Rightarrow \text{wp}(c, I)$$

Soundness (w.r.t. the modeled semantics)

The theorem that makes the verdict trustworthy — over IMP's **modeled** big-step semantics, not Python's runtime directly:

$$\{P\} c \{Q\} \iff (\forall s. P(s) \Rightarrow \text{wp}(c, Q)(s))$$

Proved in Rocq by induction on the structure of `c`. The Python-to-IMP lowering is part of the trust base (next slide).

So for a function `f(x)` with precondition `P` and postcondition `Q`, the obligation is:

$$\forall x. P(x) \Rightarrow \text{wp}(c, \lambda \text{result}. Q(x, \text{result}))$$

```
Theorem f_correct : forall x, P x -> wp c (fun result => Q x result).
```

```
Proof.
```

```
  (* discharged by the tiered pipeline *)
```

```
Qed.
```

The trust base

A proof is only as trustworthy as what it rests on. Axiomander's TCB:

Trusted (the TCB)

- **Rocq kernel** — the proof checker
- **IMP semantics + WP calculus** — proven sound *in Rocq*
- **Python → IMP lowering** — extractable from Rocq later
- **Stubs / SMT axioms** — each is an assumption

Not trusted

- **LLM-generated proofs** — re-checked by Rocq
- **Generated obligations** — mechanically derived

Warning: a single unsound stub or SMT axiom invalidates every proof that uses it — the sharpest edge of the TCB. The LLM is *untrusted*: its output is always re-checked.

Frames: calls reason from contracts

A callee's `modifies:` set is its **write frame**. Callers rely on everything outside it being preserved.

```
def inc(x: int) -> int:
    """ axiomander:
        requires: x >= 0
        modifies: none
        ensures:  result == x + 1 """
    result = x + 1
    return result

def frame_two_calls(a: int, b: int) -> int:
    """ axiomander:
        ensures: a == old(a); b == old(b); result == a + b + 2 """
    a2 = inc(a); b2 = inc(b)
    result = a2 + b2
    return result
```

The CCall rule proves $\forall v. v \notin (target :: writes) \Rightarrow v' = v$ — unlisted variables are frame-preserved.

The frame rule

This is what makes proofs **local** — the heart of composability.

$$\frac{\{P\} c \{Q\} \quad \text{modifies}(c) \cap \text{free}(R) = \emptyset}{\{P \wedge R\} c \{Q \wedge R\}}$$

- R is any assertion about variables the callee doesn't touch — it survives the call **for free**.
- The caller never unfolds the callee's body; it uses only the contract + frame.
- Axiomander generates per-callee frame lemmas so each `apply` fires independently with its own residual on failure.

The proof pipeline: three tiers

Level	Mechanism	Handles
1	deterministic Rocq (<code>wp_reduce</code> , <code>lia</code>)	assignments, conditionals, loops, frame/CCall obligations — ~80% of goals
2	SMT / Rocq-hammer (cvc4, cvc5, eprover)	linear & non-linear arithmetic, first-order residuals
2b	theory-SMT (Z3 / CVC5 <code>QF_SLIA</code>)	strings, contains/prefix, regex subsumption , float dimensions
3	rocq-piler + LLM oracle	loop invariants, induction, deep lemma chains

Cheap, deterministic tactics first. The LLM is the **fallback**, not the first resort.

Staged obligations, never one giant theorem

Never lose partial proof work. A failed attempt produces reusable artifacts, not dead ends.

- Each obligation has a **stable identifier**: `sort.insert.preserves_sorted.branch_2`.
- A failed stage emits a **residual goal** with hypotheses — exactly what the LLM needs.
- The LLM operates on the **residual proof state**, not the original program.
- Per-obligation caching, parallelisation, and fine-grained reuse all fall out of this.

```
Context:  xs_sorted : sorted(xs)
          pivot_le_head : pivot <= head(xs)
Goal:     sorted(pivot :: xs)
```

Failure is information

Two kinds of negative result, both of which feed the heal-loop. In the **decidable** / **SMT-attacked** fragment, a false goal yields a *concrete typed counterexample*:

Theory-SMT (regex)

```
def accept(p: str) -> str:
    """ axiomander:
        requires: p.re_match("[0-9]{3}")
        ensures:  result.re_match("[a-z]+") """
    result = p
    return result
```

Verdict

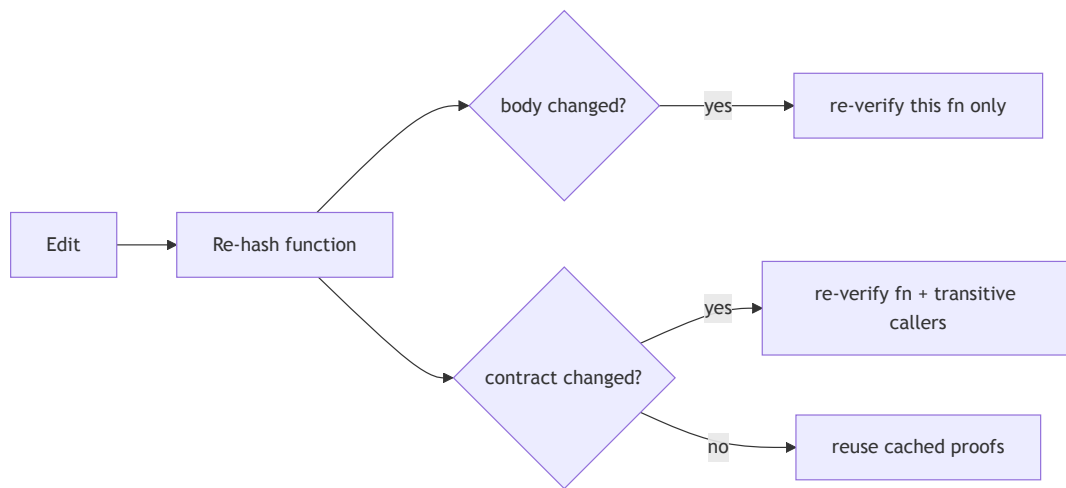
```
COUNTEREXAMPLE
  result = "000"
  satisfies requires-regex
  violates  ensures-regex
```

A digit string can never match `[a-z]+` .

Dimensional analysis is the same: mixing `[USD]` and `[GBP]` is a typed dimension error — rejected before the proof runs.

Beyond the decidable fragment, a failed proof leaves a **residual goal with hypotheses** rather than a counterexample. That residual is itself an asset — it tells the LLM *why* the spec couldn't be discharged, and is exactly what drives repair. **Future work**: proving the *negation* to certify that a contract is unsatisfiable for a given implementation.

Incremental verification = a build system for correctness



Change

Re-verify

body only

the function

local assert / invariant

the function

Dogfooding: Axiomander verifies itself

The function that decides whether a goal passed — carries a contract and proves at Level 1.

```
def is_proved(self) -> bool:
    """ axiomander:
        ensures:
            implies(self.level == ProofLevel.UNPROVED, result == False)
            implies(self.level == ProofLevel.COUNTEREXAMPLE, result == False)
            implies(self.level != ProofLevel.UNPROVED
                    and self.level != ProofLevel.COUNTEREXAMPLE, result == True)
    """
    return self.level not in (ProofLevel.UNPROVED, ProofLevel.COUNTEREXAMPLE)
```

Enum names, `implies()` per case, attribute access (`self.level`), and `not in` over a tuple — all lowered to IMP and discharged deterministically. Plus 16+ other self-verified functions.

Tooling: verification in the editor (MCP)

Axiomander is an **MCP server** — verification lives where you write code.

Tool	What it does
<code>check-function</code>	verify a single function (Level 1) + suggest contracts
<code>verify-function</code>	full verification (Level 1 $\rightarrow$ 2 $\rightarrow$ 3)
<code>verify-changed</code>	incremental — re-verify only changed functions
<code>verify-impacted</code>	dry-run — show what <i>would</i> re-verify
<code>explain-cache</code>	why a proof was reused or regenerated
<code>frame-report</code>	pre/post/invariant + frame conditions

First run compiles Rocq (seconds); subsequent runs hit the cache (milliseconds).

Part IV

What This Buys You

Threats to validity

Where this can go wrong — stated plainly:

- **Specification bugs.** Proof is *relative to the spec*; a wrong spec yields confidently-wrong code. (Mitigation: small reviewable contracts; future spec-meaning tooling.)
- **Unsound stubs / axioms.** One bad assumption invalidates downstream proofs. (Mitigation: explicit, minimal, reviewed TCB.)
- **Translation bugs.** Python → IMP lowering is trusted. (Mitigation: small verified subset; extract from Rocq later.)
- **Unsupported features.** Constructs outside the fragment are rejected, not mis-modelled — but they *are* rejected.
- **Proof-engineering cost.** Hard goals may need invariants/lemmas the LLM can't find; the loop can stall on a residual.
- **Modeled vs. runtime semantics.** We prove against IMP's model of Python, not CPython byte-for-byte.

This is like a **software bill of materials — upgraded**: instead of a list of components and hoped-for safety, we get a **precise, machine-checked trust surface** — and it is a surface we can **shrink over time** (verify a stub, discharge an axiom, extract the translator).

The argument in one slide

- LLMs are **optimistic**; they satisfy the prompt, not the intent.
- Types and tests **sample** intent; a strong specification makes it **explicit and checkable** — proof is correctness *relative to the spec*.
- Make the **specification the control surface** — iterate on it, derive code under it.
- A **heal-loop** drives the LLM on residual goals and counterexamples, not re-prompts.
- The LLM has freedom in **sub-specification**; helpers get contracts, checked the same way.
- **Composability** (frame rule + contract hashing) makes iteration and **stubbing** scale.
- The Axiomander stack — **LLM + Rocq + Iris + Python** — reduces all of this to soundly-checked proof obligations; Rocq is *one* possible backend, the methodology is backend-agnostic.

Specification is the control surface. Code is a derived artifact. Correctness requires proof.

Thank you

Axiomander 

Iterated specification management for Python.

github.com/scidonia/axiomander